

Lab 2: Conda Environments + JupyterLab + Sanity Tests (NumPy/Pandas)

Goal: Create a reproducible Python setup for this course, install core packages, verify JupyterLab, and run an automated test that also exercises NumPy/Pandas basics.

Estimated time: 60-90 minutes **Last updated:** 2026-01-27

Learning outcomes

- Create and manage a dedicated conda environment for the course.
- Install packages from conda-forge and confirm versions.
- Install and launch JupyterLab; register an ipykernel for the environment.
- Run a sanity-check script that validates imports, Jupyter commands, and basic NumPy/Pandas operations.
- Export **environment.yml** for reproducibility (use in your project repo).

Before you start

- Miniconda or Anaconda installed.
- Terminal access (Windows: Anaconda Prompt recommended; macOS/Linux: Terminal).
- Internet access for package installation.
- Create a lab folder (recommended): *geovis_labs/lab2*.

Folder setup

```
mkdir -p geovis_labs/lab2
cd geovis_labs/lab2
```

Part A - Create a course environment

1) Update conda (base)

```
conda update -n base -c defaults conda -y
```

2) Create and activate a new environment

Use a clean, modern Python (3.11 recommended).

```
conda create -n geovis python=3.11 -y
conda activate geovis
```

3) Configure conda-forge and strict channel priority (recommended)

```
conda config --add channels conda-forge
conda config --set channel_priority strict
```

4) Install required packages

```
conda install -y numpy pandas jupyterlab ipykernel
```

Optional: matplotlib (common for plots later)

```
conda install -y matplotlib
```

Part B - Register a Jupyter kernel and launch JupyterLab

1) Register the kernel (so it appears inside JupyterLab)

```
python -m ipykernel install --user --name geovis --display-name "Python (geovis)"
```

2) Launch JupyterLab

```
jupyter lab
```

In JupyterLab:

- Create a new notebook using kernel **Python (geovis)**.
- Run this cell to confirm versions:

```
import numpy as np, pandas as pd
np.__version__, pd.__version__
```

Part C - Automated sanity-check (includes NumPy/Pandas practice)

You will create a script named **check_lab2.py**. It verifies:

- You are using the correct Python executable (inside the conda environment).
- NumPy and Pandas import correctly and report versions.
- Jupyter and JupyterLab commands exist and report versions.
- A small set of NumPy operations (reshape, mean, masking).
- A small set of Pandas operations (missing values, fill, groupby summary, CSV save/reload).

1) Create **check_lab2.py** and paste the code below

```
# check_lab2.py
import sys
import platform
import subprocess
from importlib import import_module

REQUIRED = ["numpy", "pandas"]

def header(title):
    print("\n" + "=" * 70)
    print(title)
    print("=" * 70)

def run_cmd(cmd):
    try:
        out = subprocess.check_output(cmd, stderr=subprocess.STDOUT, text=True)
        return True, out.strip()
    except Exception as e:
        return False, str(e)
```

```

def check_import(pkg):
    try:
        m = import_module(pkg)
        ver = getattr(m, "__version__", "unknown")
        return True, ver
    except Exception as e:
        return False, str(e)

def numpy_pandas_smoke_tests():
    import numpy as np
    import pandas as pd

    header("NumPy smoke tests")
    a = np.arange(1, 13).reshape(3, 4)
    print("Array a:\n", a)
    print("Shape:", a.shape)
    print("Mean (axis=0):", a.mean(axis=0))
    mask = a % 2 == 0
    print("Even mask sum:", mask.sum())
    b = a * 2 + 1 # broadcasting-ish arithmetic
    print("b = a*2+1, last row:", b[-1, :])

    header("Pandas smoke tests")
    df = pd.DataFrame({
        "id": [1, 2, 3, 4, 5],
        "group": ["A", "A", "B", "B", "B"],
        "value": [10, 12, 9, None, 15]
    })
    print("DF:\n", df)
    print("\nMissing values per column:\n", df.isna().sum())

    # basic cleaning + groupby summary
    df["value_filled"] = df["value"].fillna(df["value"].median())
    summary = df.groupby("group")["value_filled"].agg(["count", "mean", "min", "max"])
    print("\nGroup summary:\n", summary)

    # save + reload to ensure IO works
    out_csv = "lab2_test_output.csv"
    df.to_csv(out_csv, index=False)
    df2 = pd.read_csv(out_csv)
    print("\nReloaded CSV head:\n", df2.head())

def main():
    header("System info")
    print("Python:", sys.version.replace("\n", " "))
    print("Executable:", sys.executable)
    print("Platform:", platform.platform())

    header("Package import checks")
    ok_all = True
    for pkg in REQUIRED:
        ok, info = check_import(pkg)
        print(f"{pkg:10s}: ", "OK" if ok else "FAIL", "|", info)
        ok_all = ok_all and ok

    header("Jupyter checks")
    ok_jup, jup_ver = run_cmd(["jupyter", "--version"])
    print("jupyter --version:", "OK" if ok_jup else "FAIL")
    print(jup_ver)

    ok_lab, lab_ver = run_cmd(["jupyter", "lab", "--version"])
    print("\njupyter lab --version:", "OK" if ok_lab else "FAIL")
    print(lab_ver)

    # Kernel list (nice-to-have; some systems may block)
    ok_k, kernels = run_cmd(["jupyter", "kernel_spec", "list"])
    print("\njupyter kernel_spec list:", "OK" if ok_k else "FAIL")
    print(kernels)

    if ok_all:

```

```

numpy_pandas_smoke_tests()
header("RESULT")
print("ð All required imports worked and smoke tests completed.")
print("Next: export your environment.yml and submit deliverables.")
else:
    header("RESULT")
    print("å One or more required imports failed.")
    print("Fix the environment, then rerun: python check_lab2.py")
    sys.exit(1)

if __name__ == "__main__":
    main()

```

2) Run the script

Expected: you see OK messages for imports and Jupyter, plus printed NumPy/Pandas outputs and a final RESULT block.

Part D - Reproducibility: export your environment

Export a clean environment file (recommended for sharing):

```
conda env export --from-history > environment.yml
```

Optional: record exact versions for debugging:

```
conda list > conda_list.txt
```

Deliverables (submit to the GeoVIZ portfolio)

- **Screenshots showing:** the results of Part A, B, C, and D
- **Short reflection** (5-8 sentences): what worked/failed, how you fixed it, and why environments matter for reproducible geovis work

Common issues and fast fixes

- **conda: command not found** - use Anaconda Prompt (Windows) or run `conda init` (macOS/Linux).
- **Imports fail** - you likely forgot `conda activate geovis`.
- **Kernel missing in Jupyter** - re-run: `python -m ipykernel install --user --name geovis --display-name "Python (geovis)"`.
- **Dependency conflicts** - confirm conda-forge + strict priority, then reinstall in a fresh environment.